API Debugging — Worked Example: Solution

> Back to the exercise · Round 1: Basic · Round 2: Intermediate · Round 3: Real-world · Round 4: Multi-agent · Hints

Story — `api_debugging_practice.py`: every round here is a bug planted in a teammate's copy of Doc02's `request_with_retry()` — the one wrapper every later document calls through for its HTTP needs. Putting each buggy function, its fix, and its test in one file lets you prove each fix works without ever touching a real network. **If not:** you'd only ever "test" these bugs by waiting for a real outage to happen, which is exactly the moment you don't have time to debug calmly.

Every fix and test below goes in `practice/api_debugging_practice.py`, and runs with `pytest api_debugging_practice.py -v` from inside `practice/`. None of the tests call the network.

- Round 1: Basic
- Round 2: Intermediate
- Round 3: Real-world
- Round 4: Multi-agent
- Real cause vs. symptom fix

Round: Basic

Real cause: in this copy, `max_attempts` and `timeout` are keyword-only (everything after the bare `*` in the signature). The caller passed `3` as a third *positional* argument, which Python won't accept — the function only takes 2 positional arguments (`method`, `url`).

Story: the error here fires before any network code even runs, so this round trains reading a `TypeError` about mismatched arguments correctly: "go check the function's signature," not "the network must be broken." **If not:** you'd start checking URLs and API keys for a bug that's entirely contained in one mistyped call line.

The fix:

```
def request_with_retry(method, url, *, max_attempts=3, timeout=(3, 5)):
    # stand-in body for practice — the real one is in http_client.py
    return {"method": method, "url": url, "max_attempts": max_attempts}

def fetch(url):
    # why: the fix is at the call site — name the setting, so it
```

```
    # can't land in the wrong slot by position
    return request_with_retry("GET", url, max_attempts=3)
```

The function's signature doesn't change — the caller does. Keyword-only arguments exist so a call can't pass a value into the wrong slot by position.

Test that would have caught it:

```
def test_fetch_calls_request_with_retry_correctly():
    result = fetch("http://example.com")
    assert result["max_attempts"] == 3
```

The buggy `fetch()` fails this test with the same `TypeError`, with no network involved — the bug is in how the function is called, so that's what the test checks.

Round: Intermediate

Real cause: the error-handling code assumed a failed response's body is always JSON, and called `.json()` on it without checking. When the final failed response is an HTML gateway error page (common for a struggling server), `.json()` itself throws — before the intended `TransientHTTPError` is ever raised. The traceback shows this inside Doc04's `ask()`, one layer above where the real bug lives (`http_client.py`) — reading from the bottom of the trace gets you to the right spot; reading from the top points at the wrong document.

Story: the error-building code itself is a place bugs can easily hide, since it only runs during a failure. Pulling "read the error body" out into its own small function makes it directly testable with a hand-made fake response object, no real server needed at all. **If not:** the only way to test this error path would be to wait for a real gateway error to happen in production.

The fix:

```
def error_body_from(response):
    # why: an error page is often HTML, not JSON — reading the
    # body must never be the thing that crashes
    try:
        return response.json()
    except ValueError:
        # how: requests' JSON error is a kind of ValueError
        return response.text
```

Inside `request_with_retry()`, the last-attempt branch becomes:

```
# a fragment — this sits inside request_with_retry()'s retry loop
if attempt == max_attempts:
    error_body = error_body_from(response)
    raise TransientHTTPError(response.status_code, error_body)
```

Test that would have caught it:

```
class FakeHtmlResponse:
    # a tiny stand-in for a requests response that holds an HTML page
    status_code = 502
    text = "<html>Bad Gateway</html>"

    def json(self):
        raise ValueError("Expecting value: line 1 column 1 (char 0)")

def test_error_body_survives_a_non_json_body():
    body = error_body_from(FakeHtmlResponse())
    assert body == "<html>Bad Gateway</html>"
```

The test has to feed a non-JSON error body on purpose — a fake that always returns clean JSON, even for errors, can never reach this path.

Round: Real-world

Real cause: `time.sleep(2 ** attempt)` has no randomness. Every caller that hits the same rate limit at the same moment waits the exact same times, so they all wake up and retry together, again and again — a "thundering herd", colliding with the same limit on every attempt. (Doc02's own `compute_backoff_delay()` already adds jitter; this copy dropped it.)

Story: the log shows three agents all retrying at the exact same second — the *pattern* across the log, not any single line in isolation, is the actual clue here. This round trains reading a log for timing patterns across multiple callers, not just scanning for an error message. **If not:** you'd just raise the retry count, and the same thundering herd would simply collide more times at the new interval.

The fix:

```
import random

def backoff_seconds(attempt):
    base = 2 ** attempt
    # why: a random extra wait, different for each caller,
    # spreads a group of retries apart instead of lockstep
```

```
    jitter = random.uniform(0, base * 0.5)
    return base + jitter
```

Test that would have caught it:

```
def test_backoff_seconds_is_not_identical_across_callers():
    values = []
    for i in range(20):
        values.append(backoff_seconds(attempt=2))
    # how: a set keeps only different values — more than one
    # means the waits really differ
    assert len(set(values)) > 1
```

One caller in isolation can't show this bug — it's about many callers at once, so the test checks that 20 "callers" don't all get the same wait.

Round: Multi-agent

Real cause: a cap on attempt *count* doesn't limit total time when the backoff keeps growing — 5 attempts can still add up to 30+ seconds of silent waiting inside one tool call, which the graph's log can't see. From the Supervisor's log alone, that looks exactly like a stuck graph, which sends debugging to the wrong layer (LangGraph edges and state, when the real problem is inside the HTTP retry wrapper).

Story: the symptom shows up in the graph's own log, but the actual cause sits two layers further down. This round trains checking the lower layer's timing configuration before assuming the bug is in the graph logic itself. Pulling the timeout rule into its own small function makes it directly testable without any real waiting involved. **If not:** a test using a fake, instant transport would keep passing cleanly, while real runs kept hanging for a real 30 seconds each time.

The fix:

```
import time

def out_of_time(start_time, max_total_seconds, now):
    # why: a hard ceiling on TOTAL wall-clock time, not just attempts
    return now - start_time > max_total_seconds

def request_with_retry(method, url, *, max_attempts=3, timeout=(3, 5),
                       max_total_seconds=15):
    start_time = time.monotonic()
    for attempt in range(1, max_attempts + 1):
        # when: checked before EVERY attempt, so a slow API fails fast
        if out_of_time(start_time, max_total_seconds, time.monotonic()):
```

```
            raise TransientHTTPError(
                None, "exceeded max_total_seconds before success"
            )
        try:
            return send_request(method, url, timeout=timeout)
        except (TimeoutError, RetryableStatusError):
            if attempt == max_attempts:
                raise
            time.sleep(backoff_seconds(attempt))
```

(`send_request` , `TransientHTTPError` and `RetryableStatusError` are the pieces your real `http_client.py` already has.) The failure now shows up fast, as an HTTP-layer error with a clear message — not as an unexplained gap in the Supervisor's log.

Test that would have caught it:

```
def test_out_of_time_stops_a_long_retry():
    # 16 seconds after starting, with a 15-second ceiling
    assert out_of_time(start_time=0, max_total_seconds=15, now=16) is True

def test_out_of_time_allows_a_quick_retry():
    assert out_of_time(start_time=0, max_total_seconds=15, now=4) is False
```

Because the time rule is its own small function, the test gives it made-up times — no sleeping, no network, and it runs in milliseconds.

Real cause vs. symptom fix

A symptom fix at any of these rounds is tempting because it's fast: wrap the call in a broader `try/except` and return `None` on any failure, or just raise `max_attempts` until the flaky test stops flaking. Both make today's failure disappear without touching why it happens — a swallowed exception just moves the missing data one layer down, to whatever code trusted a `None` it never expected, and a higher retry count just makes a thundering herd collide more times. The real-cause fixes above all make the *actual* failure condition (a non-JSON error body, retries in lockstep, unbounded total wait) impossible, not just less likely in your next test run — which is the "check layer by layer" habit from Core Concepts: the API layer's own error-handling code can have its own bugs, separate from whatever layer is calling it.